AIRO report: ontology representing an automated assembly line

Sisani Francesco, Spluga Matteo

January 31, 2026

1 Introduction

This report aims to describe the thought and building process behind the creation of an ontology to represent an automated assembly line. More specifically, the asked domain for the ontology is the construction of a circuit board given a set of components and some placement rules. The main application for this domain is the building of printed circuit boards, also known as PCBs, which are supports used to interconnect various electrical components between each other. The first concern regarding how to build the ontology was how actually the PCB construction process works: the main procedure used to build printed circuit boards is the so-called surface mount technology, or SMT for short, which allows to directly assemble the components on the board's surface. The components are also built to have the minimum weight possible, which results in large cost reductions both in terms of volume and assembly process. This new technology, along with the deploy of automated machine complexes, spiked productivity in the sector while also reducing the usage of human workforce for repetitive and very precise tasks such as the component placement and welding.

Knowing this heavy technological advance in the electronical assembly sector, the ontology displayed here aims to describe and represent an example of an SMT assembly line, providing a build with an adequate quantity of classes, object properties and data properties to represent precisely every main procedure of the proposed automated machine.

2 Cited research

Previous research that inspired and helped on the ontology's construction are cited in this section. It includes a paper that gave the baseline for representing an assembly line, a research project that gave the blueprint for actually building the ontology and an interesting approach on class organization.

2.1 Bard et al, *Machine Setup and Component Placement in Printed Circuit Board Assembly*

This paper provides a blueprint for the Fuji CP II, an automated system proposed to improve of current (1994) state of production and to contrast increasing production costs. Automatization of the assembly process was a necessity, since further research reported that over 28% of a printed circuit board's cost depended on the assembly process, hence the need to develop an automated system to reduce costs and improve productivity, flexibility and production speed.

Fuji CP II was born in a context where SMT was taking over the industry, and since the component placement operations were more precise than in the pin-through-hole strategy it was necessary to implement a system to reduce human error. The machine's blueprint is provided below:

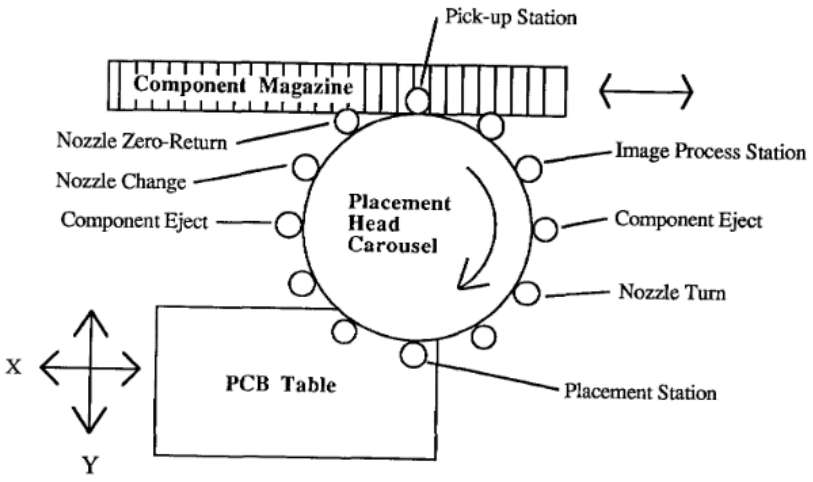


Figure 1: Fuji CP II blueprint

Figure 1 shows the main componenets of the Fuji Machine, which are:

- a rack magazine to store the components;
- a carousel composed of twelve placement heads and a clockwise spinning device;
- a moving table that moves the PCB to the coordinates where a specific component must be placed.

The machine picks up a component from the magazine, which is equipped with 100 different slots, with one placement head provided with two nozzles of different dimensions, aimed to grab different component types. The head than spins along with the carousel and it stops to a processing station, where

the grasped component is subjected to an inspection to see if some unfixable problem has occurred within the component, like broken pins or faulty wiring. If a problem is detected, the component is disposed by another station : if not, the component is placed on the already adjusted PCB.

The ontology will mainly refer to the structure of this specific assembly line, and as such it maintains the main features with some limitations for a simpler implementation.

2.2 SanFilippo et al, *Ontology-based knowledge representation for additive manufacturing*

This paper provides an ontology representation of an additive manifacturng machine, which aims to manage a big quantity of data shared by a lot of applications in an integrated way by introducing specific knowledge models. This introduction on the subject was essentially new to the sector, since ontology literature on additive manifacturing has been trailing behind other sectors such as automated machines and such.

The study provided inspiration on how to build the ontology on an entity level, mainly on the class division and organization. A branch of proposed structure is shown here for demonstration:

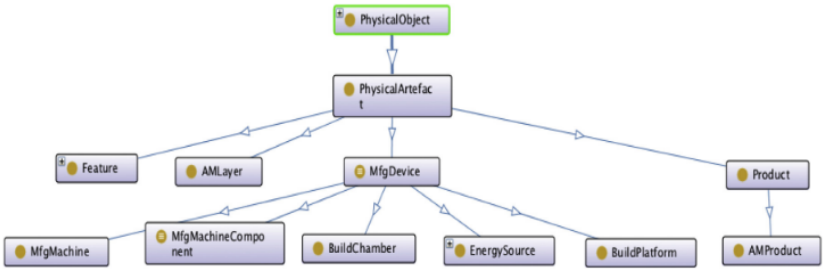


Figure 2: Demonstrative onthology branch

The approach shown in figure 2 has been the main inspiration to build this ontology for assembly machines, since it provides a way to handle different classes in a nested structure that keeps the workspace clean and organized, especially when specific machine parts are present and they need to be administered with subclasses.

3 Ontology implementation

The following subchapters define the structure of the previously described ontology, showing deeply in detail the reasoning behind the technical choices that were done.

3.1 Classes

In this section, ontology classes are listed and explained basing on the related hierarchy. Briefly explained, classes represent categories of things within the given domain considered for the construction of an ontology. Being the ontology the full model of concepts and relatioships in a domain (given a set of existing things in that domain), classes are used to group entities that share common characteristics, in order to create connections based on those characteristics between these entities.

The following is a reference image of the classes hierarchy that will be taken into account for the subsequent explanations of each one.

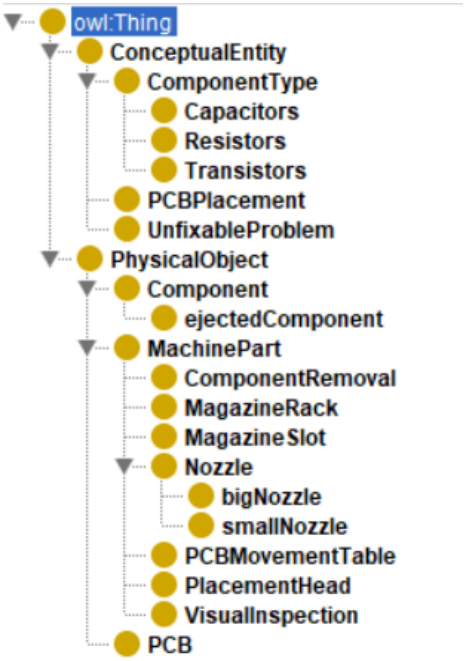


Figure 3: Classes hierarchy

In Figure 3 it can be seen the clear subdivision of classes in several macro-blocks, the biggest of them being *ConceptualEntity* and *PhysicalObject*.

3.1.1 ConceptualEntity and relative subclasses

ConceptualEntity contains all the abstract concepts that mainly represent the PCB movement (*PCBPlacement*) and robot-related problems regarding mostly common problems about its overall functionality (*UnfixableProblem*). More in detail, the *PCBPlacement* class takes into account the positioning of the

PCB during the working process, useful for the components placement, while the *UnfixableProblem* class considers that each component is examined by an automated visual inspection system for orientation and diagnostics during the movement operations. If an uncorrectable problem is identified, the component is ejected at the following station and reassigned to the end of the sequence. Otherwise, the component is oriented for placement. The *ConceptualEntity* class also contains a list of component types that need to be placed on the PCB (only considering resistors, capacitors and transistors for the sake of simplicty).

3.1.2 PhysicalObject and relative subclasses

PhysicalObject, instead, contains all the concrete entities of the ontology that describe the different components of the robot itself and the workspace physical elements it has to work with. Three main subclasses are *Component*, *MachinePart* and *PCB*.

The first generally considers the three previously described components just by using a single class. It also branches out with another subclass called *ejectedComponent* which relates to the solving process of the problem that can possibly occur during the working process (a better explanation of this process will be described in the subchapter 3.2).

The second class (*MachinePart*) describes all the components that make up the robot or the other objects it has to work with. Just by looking at the list of subclasses contained in *MachinePart*, these describe the functionality of the robot in a good way: a rotating placement head carousel, containing nozzles, picks up components from their slot in the magazine rack and positions them on the PCB. This is moved by a movement table which follows the carousel movement. This is done to let the rotating head only perform a rotation, being the movement table the component which must adjust its position in order to let the carousel correctly put the components at their place on the PCB.
In particular, the *Nozzle* class branches out with two subclasses (*bigNozzle* and *smallNozzle*) which are related to the type of component they have to grasp from the magazine rack in order to finally place them on the PCB. In fact, the presence of two grasping heads on the nozzle considers that, basing on whether the component is a resistor, a capacitor or a transistor, one of the two heads would grasp the correct component. This is purely done for a matter of component dimensions. Giving a small insight on the object properties part (described in detail in subchapter 3.2), only two properties called **grips** and **isGrippedBy** define all the reciprocal connections between the nozzle heads (big and small) having that the big nozzle only grasps capcitors or resistors (given their eventual big dimensions) while the small nozzle only grasps transistor.

This class also contains two classes which do not specifically describe physical elements, but are more performance-related. These are *VisualInspection* and *ComponentRemoval* which are specifically related to the solving process of

the potential functioning problems, as it will be seen in the next subchapter.

3.2 Object properties

In this section, which represents the core of the ontology itself, an explanation of the conceptual connections between classes is done in terms of object properties. These properties give information about the overall functioning of the robot and the different tasks it must fulfill during the working process. They allow reasoners to infer new semantic relationships between entities automatically in an ontology, enabling structured and machine-understandable links between individuals.

The following is an image of the listed object properties that were chosen for the related ontology.

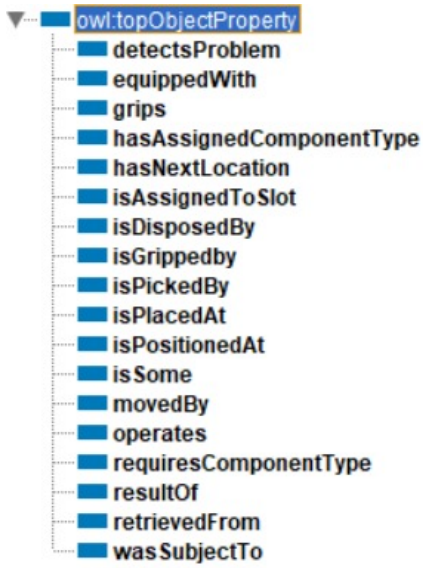


Figure 4: Object properties

All the object properties seen above are characterized with the following flags: functional, asymmetrical, irreflexive. This is done to describe the relationships between classes in the simplest way possible, even if more properties are needed for a more complete scheme. In order to simply describe in an understandable way the meaning of each one of these properties, a list containing all the related explanations is subsequently shown:

- **detectsProblem**: This is the property which allows to describe the problem detection (not the solution) with the use of sensors for a visual inspection, having that it links *VisualInspection* (Domain) with *UnfixableProblem* (Range).

The logic proposition in the classes window is "**detectsProblem** some *UnfixableProblem*".

- **equippedWith**: This is the property which better defines two elements which are grouped in the classes *MagazineRack* and *PlacementHead*. In fact, this property links *MagazineRack* (Domain) with *MagazineSlot* (Range), and *PlacementHead* (Domain) with two ranges, which are *smallNozzle* and *bigNozzle*.
  The logic propositions are "**equippedWith** max 100 *MagazineSlot*" and "**equippedWith** only (*smallNozzle* and *bigNozzle*)". The number of slots in the magazine rack was arbitrarily chosen.

- **grips**: This property connects the two nozzles manned by the placement head while also giving information about the grasping process of the three components performed by the nozzles. This property links two domains with three ranges each: *bigNozzle* and *smallNozzle* (Domains) with *Capacitors Resistors* and *Transistors* (Ranges). Here, as said in the previous chapter, a distinction on which nozzle grasps which component (basing on the physical dimensions) can be seen in the logic propositions in the classes window.
  The logic proposition lets *smallNozzle* be a subclass of "**grips** only *Transistors*" and *bigNozzle* be a subclass of "**grips** only (*Capacitors* or *Resistors*)".
  The property **grips** is the inverse of **isGrippedBy**.

- **hasAssignedComponentType**: This property explains the correlation between the slots of the magazine rack with the component it is placed in without making distinctions between the types of component. It links *MagazineSlot* (Domain) with *ComponentType* (Range).
  The logic proposition can be seen in the classes window, where *MagazineSlot* is a subclass of "**hasAssignedComponentType** max 1 *ComponentType*".

- **hasNextLocation**: This property relates to the conceptual entity *PCBPlacement* to explain the movement of the PCB during the work process of the robot. It links *PCBPlacement* (Domain) with *PCBPlacement* (Range).
  There information about the logic proposition (in the subclass section) related to this object property in the classes window was done using Data properties **hasYCoordinate** and **hasXcoordinate** by writing the phrase "(**hasXCoordinate** exactly 1 xsd:float) and (**hasYCoordinate** exactly 1 xsd:float)".

- **isAssignedToSlot**: This property expresses the position a specific component can occupy in the magazine rack, more specifically it is used to express the possibility of a component to occupy more than one slot: in the

class *ComponentType*'s SubclassOf section the query "**isAssignedToSlot** min 1 *magazineSlot*" expresses this design choice.

- **isDisposedBy**: This property gives information about the solution to th previously considered eventual problem during the work phase. In practice, when the problem is detected, the component is removed in order to start with the working flow again. In the classes window, "**isDisposedBy** only *ComponentRemoval*" describes the relation between the classes *ejectedComponent* (Domain) and *ComponentRemoval*.

- **isGrippedBy**: inverse of the object property **grips**, uses *ComponentType* as domain and *Nozzle* as range. Enlarged domains like this one have been used to cover all the instances of something happening in one function to then apply restrictions to the specific cases.

- **isPlacedAt**: This property explains where a physical component is to be placed according to the theoretical placement. As such, it has *Component* as domain and *PCBPlacement* as range and *Component* has a specific query in its SubclassOf section to express that a component has to be placed in a certain placement with specific coordinates, while not excluding more than one possible placement.

- **isPositionedAt**: It links the PCB with its position described by PCB-Placement. The logic connection is done between *PCB* (Domain) and *PCBPlacement* (Range). It is better described by "(**isPositionedAt** some (*PCBPlacement* and ((**hasXCoordinate** some xsd:float) and (**hasYCoordinate** some xsd:float)))" in the classes window.

- **isSome**: This property defines the connection between the physical component and the component type. It connects *Component* (Domain) to *ComponentType* (Range). In the classes window the relation is explained by the phrase "**isPickedBy** exactly 1 *ComponentType*".

- **movedBy**: This property simply describes the movement of the PCB performed by the movement table it is positioned on. The *PCB* domain is linked to *PCBMovementTable* range. The design choice for this property is better described by "(movedBy some PCBMovementTable)" which is connected to the previous phrase (related to the implementation of the property **isPositionedAt**) with an "and" logic connector.

- **operates**: This property explains that a placement head uses only one nozzle at a time to pick up a component. As such, it uses *PlacementHead* as domain and the nozzles as ranges and the rule is express through the phrase "operates only (bigNozzle or smallNozzle)"

- **requiresComponentType**: This links the *PCBPlacement* domain with *CompoentType* range, inferring the fact that the PCB requires components to be properly built by the robot. In the classes section, "(**requiresComponentType** some *ComponentType*)" shows the relationship between PCBPlacement and ComponentType.

- **resultOf**: This property, using *UnfixableProblem* as domain and *VisualInspection* as range, represents the inverse property of the object property *detectsProblem* and is mainly used to define the restriction between normal components and ejected components.

- **retrievedFrom**: This property expresses the action exercised on the component with respect to the component itself and the slot it was placed into before getting picked up by a nozzle. The *Component* domain is linked to the *MagazineSlot* range, while "**retrievedFrom** exactly 1 *MagazineSlot*" is the logic connection between classes.

- **wasSubjectTo**: This property expresses the visual inspection check that every component must do before being placed. As such, it is present in the SubClassOf sections of both *Component* and *ejectedComponent* to distinct between a component that passed the inspection and another that did not.

3.3 Data properties

The data properties used in the ontology are only two, defined with the names **hasXCoordinate** and **hasYCoordinate**, which are used in the PCB table motion to move the PCB on the correct position to place some specific component. As such, it occurs in the entities *PCBPlacement*, *Component* and *PCB*, with the definitions in the respective SubclassOf sections shown below:

- (*requiresComponentType* some *ComponentType*) and (**hasXCoordinate** exactly 1 xsd:float) and (**hasXCoordinate** exactly 1 xsd:float)

- **isPlacedAt** some (*PCBPlacement* and (**hasXCoordinate** some 1 xsd:float) and (**hasXCoordinate** some 1 xsd:float))

- (**isPositionedAt** some (*PCBPlacement* and (**hasXCoordinate** some 1 xsd:float) and (**hasXCoordinate** some 1 xsd:float))) and (**movedBy** some *PCBMovementTable*)

The first query, related to *PCBPlacement*, indicates that a placement refers to a specific component which has a precise position to be placed on, whereas the other two queries, related to *Component* and *PCB* respectively, ensure that the component and PCB positions are the same, while considering the possibility that more than one of the same component can be placed on a specific PCB.

3.4 Reasoner testing and debugging

After the complete definition of all the entities, a final test to verify the ontology's functionality has been made.

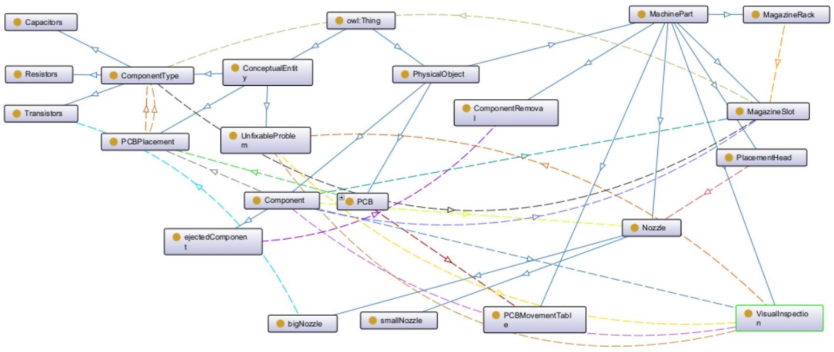


Figure 5: Entity graph obtained through the Ortograph tool

The final build has been tested with all the reasoners provided by Protégé and all of them found a consistent ontology with no owl:Nothing equivalences, with compilation times shown on the table below:

Reasoner	**Time (ms)**
ELK	529
HermiT	1727
Pellet	6

Table 1: Compilation time for different reasoners

A further step has been to test the ontology's functionality with some debugging queries, which gave positive results:

- **Query 1**: what are the components that have been marked unusable due to an unfixable problem?

  DL: Component and (wasSubjectTo some(VisualInspection and (DetectsProblem some UnfixableProblem)))

  Result: subclass *ejectedComponent*

- **Query 2**: which nozzle is capable of picking only transistors?

  DL: Nozzle and grips only Transistors

  Result: subclass *smallNozzle*

- **Query 3**: which component types are capable of being gripped only by big nozzles?

  DL: ComponentType and isGrippedBy some bigNozzle

  Result: subclasses *Resistors* and *Capacitors*

These results indicate that the presented ontology is functional and able to be used for representing an automated assembly machine.